Организация

ГЕОСКАН ПИОНЕР

МЕТОДИЧЕСКОЕ ПОСОБИЕ

Программирование светодиодной индикации и событийная модель

Автор: Фамилия И.О.

Руководитель: Фамилия И.О.

Санкт-Петербург

2026

Содержание

1 Язык Lua **3**

1.1 Введение 3
1.2 Толковый словарь 3
1.3 История и назначение 4
1.4 Lua в нашем курсе 4
1.5 Базовые конструкции Lua 4
1.6 Переменные 5
1.7 Арифметические операции 6
1.8 Логические операции 8
1.9 Условия 9
1.10 Циклы 10
1.11 Работа с модулями платформы 12
1.12 Примеры, используемые далее 12

2 Программирование светодиодов **14**

2.1 Введение 14
2.2 Толковый словарь 14
2.3 Аппаратная часть и модуль Ledbar 15
2.4 Физическая реализация 15
2.5 Особенности адресации (Буфер драйвера) 15
2.6 Теория и интерфейсы API 16
2.7 RGB-модель и диапазоны 16
2.8 Адресация и буфер 16
2.9 Модуль Ledbar 16
2.10 Таймеры 16
2.11 Событийная модель 17
2.12 Анализ подходов к программированию 18
2.13 Конечный автомат 18
2.14 Блокирующее выполнение 18
2.15 Асинхронные таймеры 19
2.16 Примеры кода 19
2.17 Проверка знаний 22
2.18 Задания для лабораторной работы 26

2.19 Базовые алгоритмы 26
2.20 Алгоритмические задачи 27
2.21 Работа с событиями 28

3 Автономные беспилотные миссии 30
3.1 Введение 30
3.2 Толковый словарь 30
3.3 Событийная модель и интерфейсы управления 31
3.4 Светодиоды для отладки миссии 32
3.5 Справочник функций полёта 32
3.6 Учебные ступени полёта 33
3.7 Геометрия траекторий: окружность 37
3.8 Архитектурные подходы 38
3.9 Разбор примеров миссий 38
3.10 Рекомендации по разработке миссий 43
3.11 Проверка знаний 45
3.12 Задания для лабораторной работы 48
3.12.1 Базовые миссии 48
3.12.2 Продвинутые задания 50
3.12.3 Формулировки продвинутых заданий 51

1 Язык Lua

1.1 Введение

Lua — лёгкий и встраиваемый язык программирования, созданный в начале 1990-х для расширения приложений скриптами. Он компактен, прост и быстро работает, поэтому широко используется в встраиваемых системах, играх и робототехнике.

1.2 Толковый словарь

Нажмите на термин в тексте, чтобы перейти к его определению.

Число вещественное значение (обычно двойной точности), например `3.14` или `10`; доступна математика через `math`.

Строка последовательность символов в кавычках, например `"Lua"`; операции: конкатенация `..`, длина `#s`.

Булево логическое значение `true` или `false`; используется в условиях.

nil «пусто»/«нет значения»; удаляет поля таблицы, означает отсутствие данных.

Таблица универсальная структура (массив и словарь одновременно); хранит позиции `t[1]` и поля `t.x`.

Функция блок кода, вызываемый по имени, принимает аргументы и возвращает значения.

type() функция, возвращающая тип значения: `type(10) -> "number"`.

Область видимости `local` делает переменную видимой только в текущем блоке; без `local` — глобальная.

Оператор символ или слово, выполняющее действие: `+ - * / % ^ and or not`.

Выражение комбинация значений и операторов, дающая новое значение.

Условие логическое выражение, определяющее выбор ветки выполнения.

Цикл повторение действий: `for`, `while`, `repeat until`.

ipairs/pairs итераторы по таблицам: `ipairs` для массивов; `pairs` для словарей (порядок не гарантирован).

Истинность в Lua ложными считаются только `false` и `nil`.

unpack распаковывает таблицу в список аргументов: `unpack({1,0,0}) -> 1,0,0`.

1.3 История и назначение

- Появился как средство «подключаемой логики» поверх приложений на C/C++. - Минимальные требования к памяти, переносимость и простой синтаксис. - Табличные структуры данных (`table`) и функции первого класса делают язык выразительным при небольшом ядре.

1.4 Lua в нашем курсе

- Мы используем Lua, чтобы программировать поведение квадрокоптера «Геоскан Пионер». - Через модуль `Ledbar` управляем светодиодной индикацией. - Через `Timer` создаём неблокирующие анимации. - Через `callback(event)` реагируем на системные события (посадка, напряжение, достижение точки).

1.5 Базовые конструкции Lua

Ниже — краткий пример базовых элементов: переменная, строка и таблица. Циклы и условия разбираются далее.

```lua
-- числовая переменная
local x = 10
-- строковая переменная
local s = "Lua"
-- таблица с позиционными элементами и именованным полем
local t = {1, 2, 3}
t.mode = "auto"
-- выводим значения и длину массива
print(x, s, #t)  -- 10  Lua  3
```

1.6 Переменные

Переменная — это имя, за которым «закреплено» значение. В Lua есть базовые типы: число, строка, булево, таблица, функция, nil. Узнать тип можно через type(). Локальная переменная объявляется через `local` и видна внутри текущего блока; **глобальная** — без `local` (видна во всём скрипте).

Число — это величина для арифметики (например, `3.14` или `10`). Строка — текст в кавычках (например, `"Pioneer"`), над ней выполняют конкатенацию `..` и получают длину `#s`. Булево — `true`/`false`, используется в условиях. Таблица — универсальная структура: массив `t[1]` и словарь `t.x` одновременно. Функция — именуемый блок кода, например `local function f(x) return x*2 end`.

```lua
-- локальная числовая переменная
local x = 10
-- глобальная строковая переменная
name = "Pioneer"
-- локальная булева переменная
local ready = true
-- таблица с позиционными элементами и именованным полем
local t = {1, 2, 3, speed = 3.5}
-- множественное присваивание
x, y = 1, 2
-- обмен значениями
x, y = y, x
-- выясняем типы значений
print(type(x), type(name), type(ready), type(t))  -- number string boolean table

-- словарь (именованные поля)
local p = {x = 1, y = 2}
-- массив (позиционные элементы)
local a = {10, 20, 30}
-- добавление нового поля
p.z = 3
-- добавление нового элемента массива
a[4] = 40
```

Пояснения. - Таблица — единственная сложная структура в Lua; через неё строят списки, словари и объекты. - `nil` при присваивании полю таблицы удаляет это поле: `t.speed = nil`. - Глобальные переменные удобно использовать только для настроек и состояний, иначе код становится хрупким; предпочитайте `local`.

Задачи для самостоятельного решения

1. Объявите локальную переменную `x` со значением `42`.
2. Присвойте строку `"Pioneer"` глобальной переменной `name`.
3. Поменяйте местами значения `a` и `b` одной строкой.
4. Создайте таблицу `t` с полем `speed=3.5` и элементами `1,2,3`.
5. Добавьте в таблицу `t` поле `mode` со значением `"auto"`.
6. Объявите `local flag = false` и затем присвойте глобальной `flag` значение `true`.
7. Создайте пустую таблицу `buf` и добавьте в неё числа `5` и `7`.
8. Удалите поле `speed` из таблицы `t`, присвоив `nil`.
9. Присвойте одновременно значения `r=1`, `g=0`, `b=0`.
10. Создайте таблицу `color` вида `{r=1,g=0,b=0}`.
11. Извлеките третий элемент массива `a` в переменную `third`.
12. Присвойте `local s = "Lua"` и `local n = 3.14`.
13. Создайте локальную функцию `id`, возвращающую свой аргумент.
14. Объявите `local config = nil` и затем присвойте `config = {enabled=true}`.
15. Создайте таблицу `point` и добавьте в неё `x=10` и `y=20`.

1.7 Арифметические операции

Операции: `+ - * / % ^` (плюс, минус, умножение, деление, остаток, возведение в степень). Сравнения: `< <= > >= == ~=`. В Lua 5.1 нет операторов целочисленного деления `//` и возведения `**`; используйте `math.floor(a/b)` и `a ^ b` соответственно. Все числа — вещественные, поэтому деление отдаёт дробные значения; для округлений применяются `math.floor`, `math.ceil` и `math.abs`.

Приоритет: сначала выполняются `^`, затем `* / %`, далее `+ -`. Скобки меняют порядок вычислений. Остаток `%` определён как «остаток от деления», полезен для задач чётности и циклических индексов.

```lua
-- приоритет: умножение выполняется раньше сложения
local a = 2 + 3 * 4
-- остаток от деления (modulo)
local r = 5 % 2
-- возведение 2 в степень 3
local p = 2 ^ 3
-- сравнение: равно?
local eq = (a == 14)
-- сравнение: больше?
local gt = (a > 10)
-- безопасное «целочисленное» деление с округлением вниз
local q = math.floor(17 / 4)  -- 4
-- евклидова длина вектора
local len = math.sqrt(3*3 + 4*4)  -- 5
```

Пояснения. - `math.sqrt` возвращает корень квадратный; `math.pi` — число π. - `math.max` и `math.min` ограничивают значения: `v = math.max(0, math.min(1, v))`. - Деление на ноль следует избегать: проверяйте знаменатель перед делением.

Задачи для самостоятельного решения

1. Вычислите выражение `3 + 4 * 2 - 5`.
2. Найдите остаток `17 % 4`.
3. Запишите `3 ^ 4`.
4. Вычислите среднее `(x + y) / 2`.
5. Запишите `math.abs(x2 - x1)`.
6. Вычислите `math.floor(n / 3)`.
7. Запишите условие «`n` чётное» с `%`.
8. Вычислите `math.sqrt(x*x + y*y)` (евклидова длина).
9. Ограничьте `v` в диапазоне `[0,1]`: `v = math.max(0, math.min(1, v))`.
10. Переведите градусы в радианы: `rad = deg * math.pi / 180`.
11. Вычислите площадь прямоугольника `S = w * h`.
12. Запишите среднее трёх чисел `(a+b+c)/3`.

13. Реализуйте линейную интерполяцию `lerp = a + t*(b-a)`.
14. Посчитайте сумму массива `a`: примените цикл `for`.
15. Найдите максимум массива `a`: пройдитесь и сравнивайте.

1.8 Логические операции

Логика: `and` (И), `or` (ИЛИ), `not` (НЕ). В Lua ложными считаются только `false` и `nil` (истинность). Операторы `and` и `or` обладают **коротким замыканием**: правая часть вычисляется только при необходимости. Сравнения работают для чисел и строк (лексикографически).

```lua
-- логическое И и НЕ
local ok = true and not false
-- «тернарный» идиом в Lua через and/or
local sel = (gt and 1 or 0)
-- композиция условий «в диапазоне»
local within = (a >= 0) and (a <= 100)
-- проверка непустой строки
local nonempty = (name ~= nil) and (name ~= "")
-- исключающее ИЛИ для булевых
local xor = (p and not q) or (q and not p)
```

Пояснения. - Идиома `(cond and A or B)` выбирает `A` при истинном `cond` и `B` иначе. - `nil` часто означает «нет данных», поэтому проверяйте значение перед доступом к полям. - Строки сравниваются лексикографически: `"a" < "b"` истинно.

Задачи для самостоятельного решения

1. Составьте условие «`x` в диапазоне `[0,1]`».
2. Верните `1` если `b==true`, иначе `0` с `and/or`.
3. Составьте проверку «строка `s` не пустая» (`s~=""`).
4. Сравните `"a"` и `"b"`: проверка «меньше» (`<`).
5. Составьте выражение «`x>0` и `y~=0`».
6. Запишите де Моргана: `not(a and b)` эквивалент `(not a) or (not b)`.

7. Реализуйте «исключающее ИЛИ» для `p` и `q`.
8. Преобразуйте число `n` в булево: `n~=0`.
9. Составьте логическое выражение: `t.enabled == true or t.mode=="auto"`.
10. Верните «сигнал опасности», если `voltage<3.5` или `temp>80`.
11. Напишите предикат «индекс чётный»: `i % 2 == 0`.
12. Составьте выражение, выбирающее максимум из `a` и `b` через `and/or`.
13. Используя `not`, инвертируйте булеву переменную `on`.
14. Составьте условие «`t` не `nil` и `t.value` существует».
15. Напишите логическую функцию «входит ли символ `c` в набор `{"r","g","b"}`».

1.9 Условия

Ветвления выполняются через `if ... then ... elseif ... else ... e`
Истинными считаются любые значения, кроме `false` и `nil`. Шаблоны: «охранные условия» (ранний выход), таблица переходов (эмуляция конвейера состояний), каскад `elseif` для диапазонов.

```
-- классификация числа x
if x > 0 then
  state = "positive"
elseif x == 0 then
  state = "zero"
else
  state = "negative"
end

-- безопасно работаем с полями, если таблица и поле существуют
if t and t.value then
  result = t.value * 2
end
```

Пояснения. - «Охранное» условие позволяет быстро завершить ветку при неверных входных данных. - Вложенные условия стоит разбивать на логические блоки для читаемости. - Для выбора из фиксированного набора значений удобно использовать таблицу функций: `actions[state]()`.

Задачи для самостоятельного решения

1. Напишите условие, присваивающее `sign="+"` если `x>0`, иначе `sign="-"`.
2. Присвойте `parity="even"` при чётном `n`, иначе `"odd"`.
3. Составьте ветвление: `temp<0` → `"cold"`, `0<=temp<20` → `"cool"`, иначе `"warm"`.
4. Присвойте `level="low"` если `v<3.7`, `"ok"` если `v>=3.7`.
5. Напишите условие: если `config.enabled` истинно, то `mode="run"`, иначе `"stop"`.
6. Реализуйте проверку: если `t` не `nil` и `t.x` существует, удвойте `t.x`.
7. Напишите ветвление, присваивающее цвет: `r=1,g=0,b=0` при `state=="RED"`, `r=1,g=1,b=0` при `"YELLOW"`, иначе `r=0,g=1,b=0`.
8. Составьте условие: если `s==""`, то `empty=true`, иначе `false`.
9. Реализуйте защиту от деления на ноль: если `b~=0`, то `q=a/b`.
10. Напишите проверку «строка `name` начинается с `"P"`» (используйте `string.sub`).
11. Если `voltage<3.5`, установите `alarm=true`.
12. Составьте ветвление: если `t.mode=="manual"` и `t.enabled`, то `run=true`.
13. Если `count==0`, присвойте `count=1`, иначе увеличьте на `1`.
14. Напишите условие, которое инвертирует булеву переменную `on`.
15. Реализуйте тройное ветвление по `speed`: `slow`, `medium`, `fast`.

1.10 Циклы

Циклы: числовой `for` (счётчик от `start` до `finish` с шагом), `while` (пока условие истинно), `repeat ... until` (выполнить хотя бы один раз). Для таблиц: ipairs (массив) и `pairs` (словарь). `break` прерывает цикл; `continue` отсутствует (можно имитировать через `goto`, но лучше структурировать условие).

```lua
-- суммирование 1..10
local sum = 0
for i = 1, 10 do
  sum = sum + i
end

-- увеличиваем x до 5
local x = 0
while x < 5 do
  x = x + 1
end
-- уменьшаем x до 0 минимум один раз
repeat
  x = x - 1
until x == 0

-- умножаем элементы массива на 2
local a = {10, 20, 30}
for i, v in ipairs(a) do
  a[i] = v * 2
end
-- увеличиваем значения словаря на 1
local t = {x=1, y=2}
for k, v in pairs(t) do
  t[k] = v + 1
end
```

Пояснения. - `ipairs` проходит по индексам 1..N без пропусков; `pairs` проходит по всем полям словаря в произвольном порядке. - В числовом `for` границы включительно: `for i=1,3 do` даёт `1,2,3`. - Для обхода диапазона индексов, используемых в светодиодной линейке, применяйте `for i=0,24 do`.

Задачи для самостоятельного решения

1. Посчитайте сумму чисел от `1` до `100`.
2. Найдите максимум в массиве `a`.
3. Подсчитайте количество чётных элементов в массиве `a`.
4. Постройте новый массив `b` из квадратов элементов `a`.
5. Разверните массив `a` (создайте `rev`).
6. Удалите из массива `a` все нули.

7. Сформируйте массив `seq` из чисел `1..20`.
8. Вычислите факториал `n` с помощью `while`.
9. Реализуйте цикл `repeat ... until`, который увеличивает `x` до `>=10`.
10. Скопируйте словарь `t` в `copy`, увеличив каждое значение на `1`.
11. Объедините два массива `a` и `b` в `c`.
12. Посчитайте, сколько значений `true` в массиве булевых `flags`.
13. Создайте таблицу `squares` с полями `["1"]=1, ["2"]=4, ..., ["10"]=100`.
14. Пройдитесь по индексам `0..24` и заполните таблицу `idx` значениями `i`.
15. Постройте массив `fibs` из первых `10` чисел Фибоначчи.

1.11 Работа с модулями платформы

- `Ledbar.new(N)` создаёт объект управления светодиодами (используем `N=25`). - `leds:set(i, r, g, b)` задаёт цвет по индексу `i` (`r,g,b` в `[0.0,1.0]`). - `Timer.new(period, fn)` создаёт периодический таймер; `Timer.callLater(delay, fn)` — отложенный вызов. - `function callback(event)` — обработчик системных событий.

1.12 Примеры, используемые далее

```lua
local unpack = table.unpack
local ledNumber = 25
local leds = Ledbar.new(ledNumber)
local function changeColor(col)
  for i=0, ledNumber-1 do
    leds:set(i, unpack(col))
  end
end
changeColor({0,1,0})
```

```lua
local ledNumber = 25
local leds = Ledbar.new(ledNumber)
for i=0, ledNumber-1 do leds:set(i, 0, 0, 0) end
Timer.callLater(3, function ()
  for i=0, ledNumber-1 do leds:set(i, 0, 0, 1) end
end)
```

```lua
local ledNumber = 25
local leds = Ledbar.new(ledNumber)
on = false
function updateBlink()
  local r = on and 1 or 0
  for i=0, ledNumber-1 do leds:set(i, r, 0, 0) end
  on = not on
end
blinkTimer = Timer.new(0.5, updateBlink)
blinkTimer:start()
```

2 Программирование светодиодов

2.1 Введение

Этот урок вводит в программирование индикации на квадрокоптере «Геоскан Пионер» на языке Lua. Мы начнём с определений, затем — теория, примеры и практические задания.

2.2 Толковый словарь

Нажмите на термин в тексте, чтобы перейти к его определению.

Модуль часть программы, отвечающая за конкретную задачу и предоставляющая функции для работы. Модуль Ledbar управляет светодиодами.

Драйвер программный компонент, который общается с аппаратурой и переводит команды в сигналы для устройств.

Буфер область памяти, куда мы записываем значения (например, цвета); драйвер считывает буфер и выводит свечение.

Переменная именованная «коробочка» со значением. Пример: `x = 10`.

Локальная/Глобальная локальная переменная объявляется через `local` и видна только внутри текущего блока; глобальная (без `local`) видна во всём скрипте.

Таймер «будильник», который вызывает указанную функцию периодически или один раз с задержкой.

Период время между двумя вызовами таймера.

Тик один вызов функции таймером.

Состояние текущее положение логики (например, «красный», «жёлтый», «зелёный»).

Конечный автомат модель, где программа находится в одном из состояний и переходит в другое при событиях.

RGB-компоненты три числа `R`, `G`, `B`, задающие яркости каналов; каждое в `[0.0, 1.0]`.

Индекс светодиода порядковый номер «лампочки» в линейке; используем `0..24`.

API набор функций модуля (`Ledbar.set`, `Timer.new` и др.).

Ledbar модуль управления RGB-светодиодами квадрокоптера.

2.3 Аппаратная часть и модуль Ledbar

2.4 Физическая реализация

На плате установлено **4 RGB-светодиода**. Каждый имеет три кристалла: красный (R), зелёный (G), синий (B). Комбинация трёх яркостей даёт итоговый цвет.

2.5 Особенности адресации (Буфер драйвера)

Важно: Ledbar работает с буфером **из 25 элементов**. Поэтому инициализация должна задавать размерность `25`.

> **Требование:** `Ledbar.new(25)`. Значение `4` использовать нельзя — диоды не загорятся.

```lua
local unpack = table.unpack
local ledNumber = 25                    -- размерность буфера
local leds = Ledbar.new(ledNumber)      -- объект управления лентой

local function changeColor(col)
  for i=0, ledNumber-1 do               -- индексы 0..24
    leds:set(i, unpack(col))            -- записываем RGB
  end
end

changeColor({0,1,0})                    -- включаем зелёный
```

2.6 Теория и интерфейсы API

2.7 RGB-модель и диапазоны

RGB — три управляющих числа: `R` (красный), `G` (зелёный), `B` (синий) в `[0.0, 1.0]`. `0.0` — канал выключен, `1.0` — горит ярко. Смешивая каналы, получаем цвета: `{1,0,0}` (красный), `{0,1,0}` (зелёный), `{0,0,1}` (синий), `{1,1,0}` (жёлтый), `{1,1,1}` (белый), `{0,0,0}` (выключено). Плавная смена значений позволяет создавать анимации.

2.8 Адресация и буфер

Индекс — позиция `0..24`. Буфер — массив из 25 ячеек, где каждая хранит цвет своего индекса. Драйвер считывает весь буфер, поэтому обновляйте все элементы, даже если видны только первые четыре.

2.9 Модуль Ledbar

- `Ledbar.new(N)` — создаёт объект управления (API); используйте `25`.
- `leds:set(i, r, g, b)` — задаёт цвет индекса `i` (`r,g,b` в `[0.0,1.0]`).
- Вспомогательная `changeColor(col)` перезаписывает все индексы.

Размерность — число элементов буфера (`25`). **Индекс** — порядковый номер, начиная с `0`.

2.10 Таймеры

- `Timer.new(period, fn)` — создаёт периодический таймер; `period` — период в секундах; `fn` вызывается каждый тик.
- `timer:start()`, `timer:stop()` — запуск/остановка таймера.
- `Timer.callLater(delay, fn)` — единичный вызов `fn` через `delay` секунд.

- Таймеры и состояния делайте глобальными (без `local`), иначе сборщик мусора их удалит.

Пример периодического таймера (мигание):

```
local ledNumber = 25
local leds = Ledbar.new(ledNumber)
local on = false
function updateBlink()
  local r,g,b = (on and 1 or 0), 0, 0
  for i=0, ledNumber-1 do leds:set(i, r, g, b) end
  on = not on
end
blinkTimer = Timer.new(0.5, updateBlink)  -- 0.5 секунды
blinkTimer:start()
```

Пример отложенного вызова (через 3 секунды):

```
local ledNumber = 25
local leds = Ledbar.new(ledNumber)
Timer.callLater(3, function ()
  for i=0, ledNumber-1 do leds:set(i, 0, 0, 1) end  -- включить синий
end)
```

2.11 Событийная модель

- `function callback(event)` — обработчик системных событий.
- Примеры: `Ev.COPTER_LANDED` (посадка), `Ev.LOW_VOLTAGE2` (низкое напряжение), `Ev.POINT_REACHED`, `Ev.ALTITUDE_REACHED`.
- Внутри `callback` быстро реагируйте на событие; не используйте `sleep()`.

Примеры:

```
local ledNumber = 25
local leds = Ledbar.new(ledNumber)
function callback(event)
  if (event == Ev.COPTER_LANDED) then
    for i=0, ledNumber-1 do leds:set(i, 0, 0, 0) end
  end
end
```

```lua
local ledNumber = 25
local leds = Ledbar.new(ledNumber)
mainTimer = Timer.new(0.1, function () end)
mainTimer:start()
function callback(event)
  if (event == Ev.LOW_VOLTAGE2) then
    mainTimer:stop()
    Timer.callLater(1, function ()
      for i=0, ledNumber-1 do leds:set(i, 1, 0, 0) end
    end)
  end
end
```

2.12 Анализ подходов к программированию

2.13 Конечный автомат

Подход основан на конечном автомате: программа находится в одном из заранее определённых состояний, а переходы между ними происходят при наступлении системных событий или по внутренней логике. Состояние хранится в переменной (например, `curr_state`), а набор действий для каждого состояния определяется таблицей функций `action`. При поступлении события (`callback(event)`) выбирается текущая функция из `action[curr_state]` и выполняется. Этот подход удобен для сценариев «включить один раз и завершить», либо для дискретных, чётко структурированных шагов. См. пример структуры в файле `4_светодиода.lua`: инициализация `Ledbar.new(25)`, установка фиксированных цветов `set(i, r, g, b)` и завершение работы.

2.14 Блокирующее выполнение

Блокирующий стиль использует задержки `sleep()`, при которых выполнение программы останавливается. На борту квадрокоптера такой подход нежелателен: блокировка может мешать обработке других задач (навигация, связь, датчики). Вместо `sleep()` следует применять асинхронные таймеры, которые вызывают функцию через заданные интервалы, не «замораживая» систему. См. файл `Светодиоды_и_таймер.lua` как пример, где последовательные `sleep` приводят к пошаговым включениям, но архитектурно безопаснее заменить их на `Timer.new` или `Timer.callLater`.

2.15 Асинхронные таймеры

Рекомендуемый подход: все анимации строятся на `Timer.new(period, fn)`. Функция `fn` выполняет «малый шаг» анимации (обновление цвета, переключение состояния), а таймер срабатывает регулярно с заданным периодом. Важно объявлять таймеры глобально, чтобы их не удалил сборщик мусора, и избегать длительных операций внутри `fn`. См. `nice_color (1).lua` как образец: глобальный таймер, небольшие приращения компонент цвета, полная перерисовка буфера `0..24` на каждом тике.

2.16 Примеры кода

```lua
-- создание порта управления светодиодом
local ledbar = Ledbar.new(25)

-- переменная текущего состояния
local curr_state = "START"

-- таблица функций, вызываемых в зависимости от состояния
action = {
  ["START"] = function(x)

    ledbar:set(0, 1, 0.0, 0.0)

    ledbar:set(1, 0.0, 1, 0.0)

    ledbar:set(2, 0.0, 0.0, 1)

    ledbar:set(3, 1, 1, 0.0)

    -- выключение двигателей и конец программы
    ap.push(Ev.ENGINES_DISARM)
    curr_state = "NONE"

  end,
}

-- функция обработки событий, автоматически вызывается автопилотом
function callback(event)
  if (event == Ev.ALTITUDE_REACHED) then
    action[curr_state]()
  end

  if (event == Ev.POINT_REACHED) then
```

```
    action[curr_state]()
  end

  if (event == Ev.COPTER_LANDED) then
    sleep(2)
    action[curr_state]()
  end
end

-- вызов функции из таблицы состояний, соответствующей первому состоянию
action[curr_state]()

-- создание порта управления светодиодом
local ledbar = Ledbar.new(25)

-- переменная текущего состояния
local curr_state = "START"

-- таблица функций, вызываемых в зависимости от состояния
action = {
  ["START"] = function(x)

    ledbar:set(0, 1, 0.0, 0.0)

    sleep(1000 / 1000.0)
    ledbar:set(1, 0.0, 1, 0.0)

    sleep(1000 / 1000.0)
    ledbar:set(2, 0.0, 0.0, 1)

    sleep(1000 / 1000.0)
    ledbar:set(3, 1, 1, 0.0)

    -- выключение двигателей и конец программы
    ap.push(Ev.ENGINES_DISARM)
    curr_state = "NONE"

  end,
}

-- функция обработки событий, автоматически вызывается автопилотом
function callback(event)
  if (event == Ev.ALTITUDE_REACHED) then
    action[curr_state]()
  end

  if (event == Ev.POINT_REACHED) then
    action[curr_state]()
  end
```

```lua
  if (event == Ev.COPTER_LANDED) then
    sleep(2)
    action[curr_state]()
  end
end

-- вызов функции из таблицы состояний, соответствующей первому состоянию
action[curr_state]()

-- https://learnxinyminutes.com/docs/ru-ru/lua-ru/ ссылка для быстрого
↪  ознакомления с основами языка LUA

-- Скрипт реализует мигание светодиодов на базовой плате случайными цветами

-- Упрощение вызова функции распаковки таблиц из модуля table
local unpack = table.unpack
-- Количество светодиодов на базовой плате
local ledNumber = 25
-- Создание порта управления светодиодами
local leds = Ledbar.new(ledNumber)

-- Функция, изменяющая цвет 4-х RGB светодиодов на базовой плате
local function changeColor(col)
    for i=0, ledNumber - 1, 1 do
        leds:set(i, unpack(col))
    end
end

-- Функция, реализующая выключение таймера и изменение цвета светодиодов на
↪  красный
local function emergency()
    -- Остановка таймера
    timerRandomLED:stop()
    -- Изменение цвета светодиодов (через секунду - т.к. после остановки таймера
    ↪  timerRandomLED
    -- его функция выполнится еще раз)
    Timer.callLater(1, function () changeColor({1, 0, 0}) end)
end

-- Функция обработки событий, автоматически вызывается автопилотом
function callback(event)
    -- Вызов функции emergency() при низком напряжении на аккумуляторе
    if (event == Ev.LOW_VOLTAGE2) then
        emergency()
    end
end

color = {0, 0, 0}
```

```lua
step = 0.05
index = 1

-- Создание таймера, каждую секунду меняющего цвета каждого из 4-х светодиодов на
↪  случайные
timerRandomLED = Timer.new(0.05, function ()
    color[index] = color[index] + step
    if color[index] > 0.95 or color[index] < 0.05 then
        if index < 3 then
            index = index + 1
        else
            index = 1
            step = -step
        end
    end
    changeColor(color)
end)
-- Запуск созданного таймера
timerRandomLED:start()
```

2.17 Проверка знаний

Перед выполнением практических заданий ответьте на вопросы. Вставьте пропущенную строчку кода или слово. Выберите один верный вариант и сверьте с пояснением.

1. Какое значение размера буфера должно быть передано в инициализацию? `leds = Ledbar.new(___)`

   (a) 4

   (b) 8

   (c) 25

   (d) 1

   Ответ: (c). Пояснение: драйвер считывает буфер из 25 элементов; значение 4 не работает.

2. Чем должно быть начальное значение счётчика в цикле? `for i = ____, ledNumber-1 do ... end`

   (a) 0

   (b) 1

(c) -1

(d) ledNumber

Ответ: (a). Пояснение: индексы линейки начинаются с 0 и заканчиваются 24.

3. Каким вызовом передать RGB-таблицу в `leds:set` при объявленном `local unpack = table.unpack`?

   (a) `leds:set(i, col)`

   (b) `leds:set(i, unpack(col))`

   (c) `leds:set(i, toRGB(col))`

   (d) `leds:set(i, rgb(col))`

   Ответ: (b). Пояснение: `unpack` распаковывает таблицу `{r,g,b}` в три аргумента.

4. Как корректно единожды включить зелёный цвет на всей линейке?

   (a) `changeColor({0,1,0})`

   (b) `changeColor({1,0,0})`

   (c) `changeColor({0,0,1})`

   (d) `changeColor({1,1,1})`

   Ответ: (a). Пояснение: компоненты RGB для зелёного — `{0,1,0}`.

5. Как создать отложенный запуск синего через 3 секунды?

   (a) `sleep(3); changeColor({0,0,1})`

   (b) `Timer.new(3, changeColor)`

   (c) `Timer.callLater(3, function() changeColor({0,0,1}) end)`

   (d) `ap.push(Ev.COPTER_LANDED)`

   Ответ: (c). Пояснение: `Timer.callLater` планирует единичный вызов без блокировки.

6. Какой период задать для мигания раз в полсекунды? `Timer.new(___, fn)`

(a) 0.5

(b) 500

(c) 50

(d) 1.5

Ответ: (a). Пояснение: период указывается в секундах.

7. Как инициализировать булевое состояние для мигания?

   (a) `on = false`

   (b) `on = 0`

   (c) `on = "false"`

   (d) `on = nil`

   Ответ: (a). Пояснение: булев тип Lua использует `true/false`.

8. Как проверить чётность индекса `i`?

   (a) `i / 2 == 0`

   (b) `i % 2 == 0`

   (c) `i == even`

   (d) `isEven(i)`

   Ответ: (b). Пояснение: оператор остатка `%` определяет чётность.

9. Как остановить запущенный таймер `t`?

   (a) `t:stop()`

   (b) `t:pause()`

   (c) `stop(t)`

   (d) `t.cancel()`

   Ответ: (a). Пояснение: API таймера предоставляет методы `start/stop`.

10. Как запустить таймер `t` после создания?

    (a) `t:run()`

    (b) `t:start()`

(c) `start(t)`

(d) `t:on()`

Ответ: (b). Пояснение: корректный метод — `start`.

11. Как называется функция-обработчик событий автопилота?

    (a) `function events(e)`

    (b) `function callback(event)`

    (c) `function onEvent(e)`

    (d) `function handler(ev)`

    Ответ: (b). Пояснение: платформа вызывает `callback` и передаёт `event`.

12. Какое событие соответствует посадке?

    (a) `Ev.POINT_REACHED`

    (b) `Ev.ALTITUDE_REACHED`

    (c) `Ev.COPTER_LANDED`

    (d) `Ev.LOW_VOLTAGE2`

    Ответ: (c). Пояснение: `COPTER_LANDED` — системное событие посадки.

13. Какое событие соответствует низкому напряжению?

    (a) `Ev.LOW_VOLTAGE2`

    (b) `Ev.BATTERY_OK`

    (c) `Ev.POWER_ON`

    (d) `Ev.MOTOR_ARMED`

    Ответ: (a). Пояснение: используйте `LOW_VOLTAGE2` для аварийной индикации.

14. Как выглядит условие для «полосы загрузки»: включить все индексы `<= k`?

    (a) `if i < k then ...`

(b) `if i <= k then ...`

(c) `if k <= i then ...`

(d) `if i == k then ...`

Ответ: (b). Пояснение: включёнными остаются все индексы не выше текущего `k`.

15. Какой допустимый диапазон значений `r,g,b` в модели RGB?

    (a) `0..255`

    (b) `0..100`

    (c) `0.0..1.0`

    (d) `-1..1`

    Ответ: (c). Пояснение: платформа принимает вещественные значения от `0.0` до `1.0`.

2.18 Задания для лабораторной работы

Используйте асинхронные таймеры, глобальные переменные для таймеров и состояний, полный проход по индексам `0..24`. Формулировки ниже предельно простые; рядом указаны подсказки.

2.19 Базовые алгоритмы

1. **Одновременное включение всей светодиодной линейки зелёным цветом.** Требуется реализовать алгоритм, обеспечивающий однократную установку зелёного цвета с компонентами `{0,1,0}` на всех светодиодах линейки с индексами от `0` до `24`. Для этого необходимо создать объект светодиодной линейки, реализовать функцию массовой установки цвета и выполнить её вызов. *Ожидаемый результат:* вся линейка равномерно светится зелёным цветом.

2. **Отложенное включение синего цвета.** Необходимо реализовать алгоритм, при котором светодиодная линейка остаётся выключенной в течение первых трёх секунд работы программы, после чего все светодиоды одновременно загораются синим цветом `{0,0,1}`. Реализация должна

использовать механизм отложенного вызова без применения блокирующих задержек. *Ожидаемый результат:* через 3 секунды после запуска программы линейка полностью загорается синим цветом.

3. **Периодическое мигание красным цветом.** Требуется реализовать периодическое переключение всей светодиодной линейки между состояниями «включено красным цветом `{1,0,0}`» и «выключено `{0,0,0}`» с периодом 0,5 с. Алгоритм должен быть основан на использовании асинхронного таймера и логической переменной состояния. *Ожидаемый результат:* равномерное мигание линейки с периодом 0,5 с.

4. **Эффект «бегущего огня» (один активный светодиод).** Необходимо реализовать алгоритм, при котором в каждый момент времени включён только один светодиод зелёного цвета `{0,1,0}`, а остальные выключены. Активный светодиод должен последовательно перемещаться от индекса `0` к `24` с циклическим повторением. *Ожидаемый результат:* визуальный эффект перемещающегося «огонька» по всей линейке.

5. **Модель светофора на основе конечного автомата.** Требуется реализовать конечный автомат с тремя состояниями: красный `{1,0,0}`, жёлтый `{1,1,0}` и зелёный `{0,1,0}`. Состояния должны циклически сменять друг друга с заданным временным интервалом. *Ожидаемый результат:* периодическая и стабильная смена цветов по схеме «красный – жёлтый – зелёный».

2.20 Алгоритмические задачи

6. **Плавная пульсация яркости красного канала (PWM-модуляция).** Необходимо реализовать алгоритм плавного увеличения и уменьшения яркости красного канала от значения `0.0` до `1.0` и обратно. Изменение яркости должно происходить малыми приращениями с использованием таймера. *Ожидаемый результат:* вся светодиодная линейка плавно пульсирует красным цветом.

7. **Стробоскопический режим (белые вспышки).** Требуется реализовать быстрое периодическое включение и выключение всей светодиодной линейки белым цветом `{1,1,1}` с малым временным интервалом.

Ожидаемый результат: частые и чёткие белые вспышки, создающие эффект стробоскопа.

8. **Отображение чётности индекса (шахматный узор).** Необходимо реализовать статический шаблон индикации, при котором светодиоды с чётными индексами светятся красным цветом `{1,0,0}`, а с нечётными — синим `{0,0,1}`. *Ожидаемый результат:* устойчивый чередующийся цветовой узор по всей линейке.

9. **Инвертируемая полицейская мигалка.** Требуется реализовать алгоритм, при котором цвета чётных и нечётных светодиодов периодически меняются местами с заданным интервалом времени. *Ожидаемый результат:* регулярное чередование цветовых паттернов, имитирующее работу полицейской световой сигнализации.

10. **Генерация цветового шума.** Необходимо реализовать алгоритм, который с заданной периодичностью устанавливает для каждого светодиода случайные значения компонент RGB в диапазоне от 0 до 1. *Ожидаемый результат:* динамично изменяющаяся цветовая картина без видимой закономерности.

2.21 Работа с событиями

11. **Реакция на событие посадки.** Требуется реализовать обработчик событий, который при получении события `Ev.COPTER_LANDED` полностью отключает светодиодную индикацию. Реализация не должна использовать блокирующие задержки. *Ожидаемый результат:* после посадки все светодиоды выключены.

12. **Аварийная индикация при низком напряжении.** Необходимо реализовать обработку события `Ev.LOW_VOLTAGE2`, при котором все активные таймеры останавливаются, а светодиодная линейка переводится в устойчивое состояние красной индикации. *Ожидаемый результат:* линейка постоянно светится красным цветом, сигнализируя об аварийном режиме.

13. **Индикатор загрузки (loading bar).** Требуется реализовать алгоритм постепенного заполнения светодиодной линейки от младших индексов

к старшим, при котором включённые светодиоды сохраняют своё состояние. *Ожидаемый результат:* визуальный эффект «заполняющейся» полосы.

14. **Сигнал SOS по азбуке Морзе.** Необходимо реализовать световую передачу сигнала «SOS» в соответствии с азбукой Морзе: три коротких импульса, три длинных импульса и снова три коротких. Длительности импульсов должны соответствовать заданным временным параметрам. *Ожидаемый результат:* корректно воспроизводимый световой сигнал SOS.

15. **Двоичный счётчик секунд.** Требуется реализовать алгоритм отображения текущего значения секунд в двоичном виде, используя светодиоды линейки как индикаторы отдельных битов. Обновление состояния должно происходить один раз в секунду. *Ожидаемый результат:* на линейке отображается двоичное представление времени в секундах.

3 Автономные беспилотные миссии

3.1 Введение

Автономная миссия — заранее запрограммированная последовательность действий квадрокоптера, выполняемая без участия оператора. В рамках этого раздела мы рассмотрим модель событий платформы «Геоскан Пионер», управление полётом через функции автопилота, реакцию на внешние события и построение базовых траекторий. Документ ориентирован на академический стиль и детально объясняет каждый термин, опираясь на толковый словарь и развёрнутые примеры.

3.2 Толковый словарь

Автопилот (`ap`) программный интерфейс управления полётом: команды взлёта, посадки, полёта к точке, обновления курса.

События (`Ev.*`) сигналы от платформы: `TAKEOFF_COMPLETE` (взлёт завершён), `POINT_REACHED` (точка достигнута), `COPTER_LANDED` (посадка), `SHOCK` (удар/столкновение), `LOW_VOLTAGE2` (низкое напряжение), `ENGINES_DISARM` (выключение двигателей), `MCE_*` (команды режима).

Обработчик событий (`callback(event)`) функция, которую платформа вызывает при наступлении системного события.

Таймер (`Timer`) механизм неблокирующих вызовов: `Timer.new(period, fn)` — периодический, `Timer.callLater(delay, fn)` — отложенный.

Локальная система координат плоскость X–Y и высота Z относительно текущей базы; единицы — метры.

Курс (`yaw`) угловое направление, выражаемое в радианах; конвертация градусов: `rad = deg * math.pi / 180`.

Траектория набор точек или правило формирования положения во времени (например, окружность).

Конечный автомат разбиение логики на состояния с переходами по событиям; состояние хранится в переменной.

RC-каналы (`Sensors.rc`) чтение тумблеров/органов управления для запуска миссии или изменения режима.

Индикаторы (Ledbar) световая индикация состояния миссии, аварий и этапов.

3.3 Событийная модель и интерфейсы управления

- Предстартовая подготовка: `ap.push(Ev.MCE_PREFLIGHT)` — перевод автопилота в безопасный режим подготовки. Проверяются датчики, параметры и готовность; коптер не движется. Этот шаг запускают первым, чтобы корректно инициировать взлёт.

- Взлёт: `ap.push(Ev.MCE_TAKEOFF)` — команда на подъём до штатной высоты `Flight_com_takeoffAlt`. По достижению высоты платформа генерирует событие `Ev.TAKEOFF_COMPLETE`.

- Посадка: `ap.push(Ev.MCE_LANDING)` — команда на снижение и касание. По завершении посадки генерируется `Ev.COPTER_LANDED`.

- Выключение двигателей: `ap.push(Ev.ENGINES_DISARM)` — выключение моторов (обычно после `COPTER_LANDED` или при аварии).

- Полёт к точке: `ap.goToLocalPoint(x, y, z)` — установка целевой точки в локальной системе координат (метры). По достижению цели генерируется `Ev.POINT_REACHED`.

- Обновление курса: `ap.updateYaw(rad)` — установка курса в радианах; для градусов применяют перевод `rad = deg * math.pi / 180`.

- Считывание RC: `Sensors.rc()` — получение массива каналов радиоуправления; для тумблера `SwA` обычно анализируют 8-й канал (`rc_chans[8]`).

- Обработчик событий: `function callback(event)` — функция, вызываемая платформой при наступлении `Ev.*` (`TAKEOFF_COMPLETE`, `POINT_REACHED`, `COPTER_LANDED`, `SHOCK`, `LOW_VOLTAGE2`). Внутри `callback` строятся переходы автомата миссии.

- Таймеры: `Timer.callLater(delay, fn)` — безопасная «задержка» без блокировки потока; `Timer.new(period, fn)` — периодическое действие через фиксированный интервал (анимации, траектории).

- Принципы неблокирующей логики: избегать `sleep()`; строить сценарий через события и таймеры, чтобы обработчик `callback` оставался кратким.

3.4 Светодиоды для отладки миссии

Светодиодная индикация — простой способ визуально подтверждать прохождение этапов миссии: изменение цвета служит отметкой «состояние автомата изменилось». Это помогает отладке без логов: по текущему цвету можно понять, выполнены ли *предстарт*, *взлёт*, *полет*, *посадка* или *аварийная реакция*. В учебных примерах далее сосредотачиваемся только на полёте; использование линейки оставляем как вспомогательный инструмент для реальных испытаний.

3.5 Справочник функций полёта

- `ap.push(Ev.MCE_PREFLIGHT)` — вводит борт в режим безопасной предстартовой подготовки. Проверяются датчики и параметры, движение не выполняется. Рекомендуется всегда инициировать этот шаг первым.

- `ap.push(Ev.MCE_TAKEOFF)` — команда взлёта до штатной высоты `Flight_com_takeoffAlt`. По достижении высоты платформа генерирует событие `Ev.TAKEOFF_COMPLETE`.

- `ap.push(Ev.MCE_LANDING)` — команда на снижение и касание. Завершение посадки сопровождается событием `Ev.COPTER_LANDED`.

- `ap.push(Ev.ENGINES_DISARM)` — выключение двигателей. Применяется после `COPTER_LANDED` или при аварийных ситуациях.
- `ap.goToLocalPoint(x, y, z)` — установка целевой точки в локальной системе координат (метры); оси: X — вперёд/назад, Y — влево/вправо, Z — высота. По достижении цели генерируется `Ev.POINT_REACHED`.
- `ap.updateYaw(rad)` — установка курса в радианах. Для значений в градусах используйте преобразование `rad = deg * math.pi / 180`.
- `Timer.callLater(delay, fn)` — отложенный неблокирующий вызов функции `fn` через `delay` секунд. Удобен для последовательностей «после события — действие».
- `Timer.new(period, fn)` — периодический таймер; функция `fn` вызывается каждые `period` секунд, что позволяет организовать пошаговое обновление траектории/индикации.
- `function callback(event)` — центральный обработчик событий платформы. Внутри `callback` реализуются переходы автомата миссии и вызываются соответствующие действия.

3.6 Учебные ступени полёта

Предисловие. Итак, мы готовы приступить к программированию квадрокоптера. В этом разделе последовательно пройдём ключевые этапы: от безопасной предстартовой подготовки до взлёта, полёта по заданным точкам и посадки. Каждый пример предваряется кратким пояснением, а строки кода снабжены комментариями, чтобы было понятно, что делает каждая команда.

Только запуск двигателей

```
ap.push(Ev.MCE_PREFLIGHT)                -- безопасная предстартовая подготовка
Timer.callLater(1, function()            -- отложить действие на 1 секунду
  ap.push(Ev.ENGINES_ARM)                -- команда запуска двигателей (без
  ↪ взлёта)
end)
```

Пояснение. На практике запуск двигателей выполняется в составе взлёта (`Ev.MCE_TAKEOFF`). Явная команда `Ev.ENGINES_ARM` полезна для проверки работы моторов без взлёта.

Запуск, взлёт, посадка

```
ap.push(Ev.MCE_PREFLIGHT)                -- предстарт
Timer.callLater(1, function()            -- через 1 секунду
  ap.push(Ev.MCE_TAKEOFF)                -- инициировать взлёт
end)

function callback(event)                  -- обработчик системных событий
  if event == Ev.TAKEOFF_COMPLETE then    -- взлёт завершён
    ap.push(Ev.MCE_LANDING)               -- выполнить посадку
  end
  if event == Ev.COPTER_LANDED then       -- посадка завершена
    ap.push(Ev.ENGINES_DISARM)            -- выключить двигатели
  end
end
```

Пояснение. В обработчике `callback` реагируем на завершение взлёта и инициируем посадку. После касания — выключаем двигатели.

Запуск, взлёт, полёт по прямой, посадка

```
ap.push(Ev.MCE_PREFLIGHT)                -- предстарт
Timer.callLater(1, function()            -- через 1 секунду
  ap.push(Ev.MCE_TAKEOFF)                -- взлёт
end)

local unpack = table.unpack              -- сокращение доступа к распаковке
local target = {1.0, 0.0, 0.8}           -- целевая точка (x,y,z)

function callback(event)                  -- обработчик событий
  if event == Ev.TAKEOFF_COMPLETE then    -- после взлёта
    ap.goToLocalPoint(unpack(target))     -- команда полёта к точке
  end
  if event == Ev.POINT_REACHED then       -- точка достигнута
    ap.push(Ev.MCE_LANDING)               -- выполнить посадку
  end
end
```

Пояснение. После взлёта выполняем полёт к точке `(x=1.0,y=0.0,z=0.8)`. По событию `POINT_REACHED` — посадка.

Полет по фигуре (набор точек)

```
local unpack = table.unpack                -- сокращение для распаковки
local points = {                           -- набор точек (маршрут)
  {0.0, 0.5, 0.8}, {0.5, 0.5, 0.8}, {0.5, 0.0, 0.8}, {0.0, 0.0, 0.8}
}
local curr = 1                             -- индекс текущей точки

local function nextPoint()                 -- функция перехода к следующей точке
  if curr <= #points then                  -- если есть непройденные точки
    ap.goToLocalPoint(unpack(points[curr])) -- команда полёта
    curr = curr + 1                        -- увеличить индекс
  else
    ap.push(Ev.MCE_LANDING)                -- иначе посадка
  end
end

ap.push(Ev.MCE_PREFLIGHT)                  -- предстарт
Timer.callLater(1, function()              -- через 1 секунду
  ap.push(Ev.MCE_TAKEOFF)                  -- взлёт
end)

function callback(event)                    -- обработчик событий
  if event == Ev.TAKEOFF_COMPLETE then nextPoint() end -- старт маршрута
  if event == Ev.POINT_REACHED then nextPoint() end    -- шаг по достижению точки
end
```

Пояснение. Идиоматичный «пошаговый» обход точек: каждое `POINT_REACHED` инициирует переход к следующей цели.

Полет по треугольнику

```
local unpack = table.unpack                -- распаковка таблицы
local r = 0.6                              -- радиус описанной окружности
local z = 0.9                              -- высота полёта
local angles = {0, 120, 240}               -- углы вершин (градусы)
local points = {}                          -- массив координат вершин
for i=1, #angles do                        -- вычисление координат
  local a = angles[i] * math.pi / 180      -- радианы
  points[i] = {r*math.cos(a), r*math.sin(a), z} -- (x,y,z) вершины
end
local curr = 1                             -- текущая вершина
local function nextPoint()                 -- переход к следующей
  if curr <= #points then                  -- если есть вершины
    ap.goToLocalPoint(unpack(points[curr])) -- команда полёта
    curr = curr + 1                        -- увеличить индекс
  else
    ap.push(Ev.MCE_LANDING)                -- завершение — посадка
```

```
  end
end
ap.push(Ev.MCE_PREFLIGHT)                    -- предстарт
Timer.callLater(1, function() ap.push(Ev.MCE_TAKEOFF) end) -- взлёт
function callback(event)                      -- обработчик событий
  if event == Ev.TAKEOFF_COMPLETE then nextPoint() end     -- первый шаг
  if event == Ev.POINT_REACHED then nextPoint() end        -- шаг по событию
end
```

Пояснение. Вершины равностороннего треугольника расположены на окружности: углы 0°, 120°, 240°. Переводим градусы в радианы и вычисляем координаты по `x=r*cos(a), y=r*sin(a)`.

Полет по квадрату

```
local unpack = table.unpack                  -- распаковка таблиц
local a = 1.0                                -- длина стороны квадрата
local z = 0.9                                -- высота полёта
local h = a/2                                -- половина стороны (координаты ±h)
local points = {                             -- вершины квадрата по часовой стрелке
  {-h, -h, z}, {h, -h, z}, {h,  h, z}, {-h,  h, z}
}
local curr = 1                               -- индекс текущей вершины
local function nextPoint()                   -- переход к следующей вершине
  if curr <= #points then                    -- если есть вершины
    ap.goToLocalPoint(unpack(points[curr])) -- полёт к вершине
    curr = curr + 1                          -- увеличить индекс
  else
    ap.push(Ev.MCE_LANDING)                  -- завершение — посадка
  end
end
ap.push(Ev.MCE_PREFLIGHT)                    -- предстарт
Timer.callLater(1, function() ap.push(Ev.MCE_TAKEOFF) end) -- взлёт
function callback(event)                      -- обработчик событий
  if event == Ev.TAKEOFF_COMPLETE then nextPoint() end     -- первый шаг
  if event == Ev.POINT_REACHED then nextPoint() end        -- шаг по достижению
end
```

Пояснение. Квадрат центрирован в начале координат. Его вершины расположены в точках

$$\left(-\frac{a}{2}, -\frac{a}{2}, z\right), \left(\frac{a}{2}, -\frac{a}{2}, z\right), \left(\frac{a}{2}, \frac{a}{2}, z\right), \left(-\frac{a}{2}, \frac{a}{2}, z\right),$$

что соответствует стороне длины a и фиксированной высоте z. Центр квадрата — в начале координат $(0, 0, z)$.

Полёт по окружности с поворотом квадрокоптера

```
local value = 3                          -- шаг приращения угла (градусы)
local i = 0                              -- текущий угол для курса
angleT = Timer.new(0.1, function()       -- таймер обновления курса
  ap.updateYaw(-i/180*math.pi)           -- курс в радианах (из градусов i)
  i = i + value                          -- приращение угла курса
end)

local r = 0.3                            -- радиус окружности
local angle = 0                          -- текущий угол точки (градусы)
local height = 0.7                       -- высота полёта
pointT = Timer.new(0.1, function()       -- таймер обновления точки
  angle = (angle + value) % 360          -- приращение угла точки
  local x = r*math.cos(angle * math.pi / 180) -- X по окружности
  local y = r*math.sin(angle * math.pi / 180) -- Y по окружности
  ap.goToLocalPoint(x, y, height)        -- команда на точку
end)

ap.push(Ev.MCE_PREFLIGHT)                -- предстарт
Timer.callLater(2, function ()           -- через 2 секунды
  ap.push(Ev.MCE_TAKEOFF)                -- взлёт
end)

function callback(event)                  -- обработчик событий
  if event == Ev.TAKEOFF_COMPLETE then    -- после взлёта
    angleT:start()                        -- запуск таймера курса
    pointT:start()                        -- запуск таймера точки
    Timer.callLater(6, function ()        -- завершить через 6 секунд
      pointT:stop()                       -- остановить таймер точки
      angleT:stop()                       -- остановить таймер курса
      ap.goToLocalPoint(0, 0, height)     -- вернуться в центр
      ap.push(Ev.MCE_LANDING)             -- посадка
    end)
  end
end
```

Пояснение. Два таймера управляют курсом и точкой окружности. Только неблокирующие обновления; завершение — по отложенному действию.

3.7 Геометрия траекторий: окружность

Окружность радиуса r в плоскости X–Y задаётся параметрически:

$$x(\theta) = r\cos\theta, \quad y(\theta) = r\sin\theta, \quad z = \text{const}.$$

Здесь θ — параметр угла в радианах. Если исходные значения заданы в градусах θ_{deg}, используйте перевод в радианы

$$\theta_{\text{rad}} = \theta_{\text{deg}} \cdot \frac{\pi}{180}.$$

Высота z выбирается постоянной для горизонтального полёта; курс `yaw` может сопровождаться как функция θ через `ap.updateYaw`.

3.8 Архитектурные подходы

- Конечный автомат: состояния подготовки, полёта и посадки; действия оформлены как таблица функций; переходы выполняются по событиям в `callback`.
- Асинхронные таймеры: пошаговое обновление логики без блокировок; одноразовые и периодические вызовы через `Timer.callLater` и `Timer.new`.
- Избегание блокировок: не использовать `sleep()` в сценариях полёта; для задержек применять таймеры и переходы по событиям.
- Безопасность: по `Ev.SHOCK` немедленно остановить таймеры и двигатели, включить аварийную индикацию; по `Ev.LOW_VOLTAGE2` выполнить безопасную посадку или перейти в аварийный режим.

3.9 Разбор примеров миссий

Урок 12: окружность по точкам

```
local ledNumber = 4                          -- количество видимых диодов на плате
local leds = Ledbar.new(ledNumber)          -- объект управления светодиодами
local unpack = table.unpack                 -- сокращение для распаковки таблиц
local curr_state = "PREPARE_FLIGHT"         -- начальное состояние автомата

local function changeColor(color)           -- функция смены цвета всех диодов
  for i=0, ledNumber - 1 do                 -- проход по индексам 0..(ledNumber-1)
    leds:set(i, unpack(color))              -- установка RGB для индекса i
  end
end

local colors = {                            -- палитра используемых цветов
  {1,0,0}, {1,1,1}, {0,1,0}, {1,1,0},
```

```
  {1,0,1}, {0,0,1}, {0,0,0}
}

local r = 0.3                              -- радиус окружности
local angle = 30                           -- угловой шаг (градусы)
local points = {}                          -- массив точек окружности
for i=1, 12 do                             -- генерация 12 равномерных точек
  xCoord = r * math.cos(i * angle * math.pi / 180) -- X координата
  yCoord = r * math.sin(i * angle * math.pi / 180) -- Y координата
  points[i] = {xCoord, yCoord, 0.7}        -- точка с фиксированной высотой z=0.7
end

local j = 1                                -- индекс текущей точки
action = {                                 -- таблица функций по состояниям
  ["PREPARE_FLIGHT"] = function()          -- подготовка полёта
    changeColor(colors[2])                 -- белый — подготовка
    Timer.callLater(2, function () ap.push(Ev.MCE_PREFLIGHT) end) -- предстарт
    Timer.callLater(4, function () changeColor(colors[3]) end)    -- зелёный —
    ↪ готовность
    Timer.callLater(6, function ()        -- через 6 секунд
      ap.push(Ev.MCE_TAKEOFF)             -- взлёт
      curr_state = "FLIGHT"               -- переход к полёту
    end)
  end,
  ["FLIGHT"] = function ()                 -- состояние полёта
    while j < #points do                   -- цикл одного шага по точке
      ap.goToLocalPoint(unpack(points[j])) -- полёт к текущей точке
      j = j + 1                            -- увеличить индекс
      curr_state = "FLIGHT"                -- остаться в состоянии
      break                                -- выйти из цикла (один шаг)
    end
    curr_state = "PIONEER_LANDING"         -- переход к посадке, когда точки
    ↪ завершены
  end,
  ["PIONEER_LANDING"] = function ()        -- состояние посадки
    changeColor(colors[6])                 -- синий — посадка
    Timer.callLater(2, function () ap.push(Ev.MCE_LANDING) end) -- команда посадки
  end
}

function callback(event)                   -- обработчик системных событий
  if (event == Ev.TAKEOFF_COMPLETE) then action[curr_state]() end -- старт логики
  if (event == Ev.SHOCK) then              -- авария (столкновение)
    changeColor(colors[1])                 -- красный цвет
    ap.push(ENGINES_DISARM)                -- выключение двигателей (требуется
    ↪ Ev.ENGINES_DISARM)
  end
```

```
  if (event == Ev.POINT_REACHED) then action[curr_state]() end -- переход по
  ↪ достижению точки
  if (event == Ev.COPTER_LANDED) then changeColor(colors[7]) end -- выключить
  ↪ индикацию
end

changeColor(colors[1])                    -- начальная индикация (красный)
Timer.callLater(2, function () action[curr_state]() end) -- запуск автомата через
↪ 2с
```

Комментарий. Генерация 12 точек окружности, переходы по событиям. Рекомендуется заменить `while/break` на явный шаг через таймер или по событию `POINT_REACHED`. Команда `ap.push(ENGINES_DISARM)` должна использовать пространство имён событий: `ap.push(Ev.ENGINES_DISARM)`.

Урок 14: непрерывная окружность и курс

```
local ledNumber = 4                        -- количество диодов
local leds = Ledbar.new(ledNumber)        -- объект управления линейкой
local unpack = table.unpack               -- распаковка таблиц
local curr_state = "PREPARE_FLIGHT"       -- начальное состояние

local function changeColor(color)         -- смена цвета всех диодов
  for i=0, ledNumber - 1 do               -- индексы 0..(ledNumber-1)
    leds:set(i, unpack(color))            -- установить RGB
  end
end

local colors = {                          -- палитра
  {1,0,0}, {1,1,1}, {0,1,0}, {1,1,0},
  {1,0,1}, {0,0,1}, {0,0,0}
}

local value = 3                           -- шаг увеличения угла (градусы)
local i = 0                               -- текущий угол для yaw
angleT = Timer.new(0.1, function()        -- таймер курса
  ap.updateYaw(-i/180*math.pi)            -- курс в радианах
  i = i + value                           -- приращение угла курса
end)

local r = 0.3                             -- радиус окружности
local angle = 0                           -- текущий угол точки
local xCord = 0                           -- текущая X координата
local yCord = 0                           -- текущая Y координата
local height = 0.7                        -- высота полёта
pointT = Timer.new(0.1, function()        -- таймер точки
```

```
  angle = angle + value                    -- приращение угла
  if angle > 360 then angle = angle - 360 end -- нормализация
  yCord = r*math.sin(angle * math.pi / 180)   -- расчёт Y
  xCord = r*math.cos(angle * math.pi / 180)   -- расчёт X
  ap.goToLocalPoint(xCord, yCord, height)  -- команда полёта
end)

action = {                                 -- таблица состояний
  ["PREPARE_FLIGHT"] = function(x)         -- подготовка полёта
    changeColor(colors[2])                 -- белый
    Timer.callLater(2, function () ap.push(Ev.MCE_PREFLIGHT) end) -- предстарт
    Timer.callLater(4, function () changeColor(colors[3]) end)    -- зелёный
    Timer.callLater(6, function ()        -- через 6 секунд
      ap.push(Ev.MCE_TAKEOFF)             -- взлёт
      ap.goToLocalPoint(0,0,height)       -- выход в центр
      curr_state = "FLIGHT_TO_FIRST_POINT" -- переход к полёту
    end)
  end,
  ["FLIGHT_TO_FIRST_POINT"] = function (x) -- полёт к первой точке
    changeColor(colors[4])                 -- жёлтый
    Timer.callLater(1, function ()         -- через 1 секунду
      angleT:start()                       -- запуск курса
      pointT:start()                       -- запуск траектории
      sleep(6)                             -- блокирующая задержка (не
      ↪  рекомендуется)
      curr_state = "PIONEER_LANDING"       -- переход к посадке
    end)
  end,
  ["PIONEER_LANDING"] = function (x)       -- состояние посадки
    changeColor(colors[2])                 -- белый
    Timer.callLater(2, function ()         -- через 2 секунды
      pointT:stop()                        -- остановить таймер точки
      angleT:stop()                        -- остановить таймер курса
      ap.goToLocalPoint(0, 0, height)      -- возврат в центр
      ap.push(Ev.MCE_LANDING)              -- посадка
    end)
  end
}

function callback(event)                   -- обработчик системных событий
  if (event == Ev.TAKEOFF_COMPLETE) then action[curr_state]() end -- запуск логики
  if (event == Ev.SHOCK) then              -- авария
    changeColor(colors[1])                 -- красный
    angleT:stop(); pointT:stop()           -- останов таймеров
  end
  if (event == Ev.POINT_REACHED) then action[curr_state]() end -- шаг логики
  if (event == Ev.COPTER_LANDED) then changeColor(colors[7]) end -- выключение
  ↪  индикации
```

```
end

changeColor(colors[1])                    -- начальный красный цвет
Timer.callLater(2, function () action[curr_state]() end) -- старт автомата через
↪  2с
```

Комментарий. Реализована непрерывная подача команды на полёт по окружности и обновление курса. Следует заменить `sleep(6)` на безопасную последовательность таймеров или переходы по событиям, чтобы не блокировать обработку задач.

RC-инициация миссии

```
local unpack = table.unpack               -- распаковка таблицы
local rc = Sensors.rc                     -- чтение RC-каналов
local points = {{0,2,1},{0,1,1},{0,2,1},{0,1,1}} -- маршрут по точкам
local curr_point = 1                      -- индекс текущей точки

local function nextPoint()                -- переход к следующей точке
  if(curr_point <= #points) then          -- если есть точки впереди
    Timer.callLater(1, function()         -- задержка перед командой
      ap.goToLocalPoint(unpack(points[curr_point])) -- команда полёта
      curr_point = curr_point + 1         -- увеличить индекс
    end)
  else
    Timer.callLater(1, function() ap.push(Ev.MCE_LANDING) end) -- посадка
  end
end

function callback(event)                  -- обработчик событий
  if(event == Ev.TAKEOFF_COMPLETE) then Timer.callLater(5, nextPoint) end -- старт
  if(event == Ev.POINT_REACHED) then Timer.callLater(5, nextPoint) end    -- шаг
end

startTimer = Timer.new(1, function()      -- опрос RC каждую секунду
  rc_chans = table.pack(rc())             -- считанные каналы
  if rc_chans[8] > 0 then                 -- тумблер включён
    curr_point = 1                        -- сброс маршрута
    ap.push(Ev.MCE_PREFLIGHT)             -- предстарт
    Timer.callLater(6, function() ap.push(Ev.MCE_TAKEOFF) end) -- взлёт
  end
end)
startTimer:start()                        -- запуск опроса RC
```

Комментарий. Запуск миссии по положению тумблера, отложенные переходы по событиям. Рекомендуется добавлять аварийные реакции на `Ev.SHOCK` и `Ev.LOW_VOLTAGE2`.

Локальные точки и индикация

```
local boardNumber = boardNumber            -- номер борта (если используется)
local unpack = table.unpack                -- распаковка таблиц
local points = {                           -- заранее заданные точки
  {-0.6, 0.3, 0.2}, {0.6, 0.3, 0.2}, {0, 0, 0.5}, {0.6, -0.3, 0.2}
}
local curr_point = 1                       -- текущая точка

local function nextPoint()                 -- переход к следующей точке
  if(#points >= curr_point) then           -- есть непройденные точки
    ap.goToLocalPoint(unpack(points[curr_point])) -- полёт к точке
    curr_point = curr_point + 1            -- увеличить индекс
  else
    ap.push(Ev.MCE_LANDING)                -- завершение — посадка
  end
end

function callback(event)                   -- обработчик событий
  if(event == Ev.TAKEOFF_COMPLETE) then nextPoint() end    -- старт маршрута
  if(event == Ev.POINT_REACHED) then nextPoint() end       -- шаг по событию
end

local leds = Ledbar.new(25)                -- объект линейки (буфер 25)
local blink = 0                            -- состояние мигания
for i=0,24 do leds:set(i,1,1,1) end        -- начальный белый цвет
timerBlink = Timer.new(1, function ()      -- таймер мигания, период 1с
  blink = (blink == 1) and 0 or 1          -- переключение состояния
  for i=0,24 do leds:set(i, blink, 0, 0) end -- мигание красным
end)
timerBlink:start()                         -- запуск мигания
ap.push(Ev.MCE_PREFLIGHT)                  -- предстарт
Timer.callLater(1, function() ap.push(Ev.MCE_TAKEOFF) end) -- взлёт
```

Комментарий. Для корректной индикации используйте буфер размером 25, полный проход по индексам 0..24.

3.10 Рекомендации по разработке миссий

- Делайте таймеры и состояния глобальными; избегайте локальных объектов в краткоживущих функциях.
- Исключайте блокировки (`sleep()`); опирайтесь на `Timer` и событийную обработку `callback`.
- Чётко определяйте состояния автомата; держите обработчики событий короткими.

- Реализуйте аварийные реакции и визуальную индикацию этапов.
- Работайте с курсом `yaw` в радианах; переводите градусы по формуле $\theta_{\text{rad}} = \theta_{\text{deg}} \cdot \pi/180$.

3.11 Проверка знаний

1. Как инициировать предстартовую подготовку? `ap.push(___)`

   (a) `Ev.MCE_PREFLIGHT`

   (b) `Ev.PREFLIGHT`

   (c) `Ev.START`

   (d) `Ev.PRE`

   Ответ: (a). Пояснение: используется пространство имён `Ev.MCE_*`.

2. Чем реагировать на завершение взлёта?

   (a) `callback(Ev.TAKEOFF_COMPLETE)`

   (b) `function callback(event) if event==Ev.TAKEOFF_COMPLETE then ... end end`

   (c) `ap.onTakeoff()`

   (d) `Timer.new(5, ...)`

   Ответ: (b). Пояснение: события обрабатываются внутри `callback`.

3. Как задать полёт к точке?

   (a) `ap.goToLocalPoint(x,y,z)`

   (b) `ap.fly(x,y,z)`

   (c) `ap.moveTo(x,y,z)`

   (d) `ap.goto(x,y,z)`

   Ответ: (a). Пояснение: корректный API платформы.

4. Как обновить курс по градусам `deg`?

   (a) `ap.updateYaw(deg)`

   (b) `ap.updateYaw(deg*math.pi/180)`

   (c) `ap.updateYaw(deg/180)`

   (d) `ap.updateYaw(math.pi*deg)`

   Ответ: (b). Пояснение: `updateYaw` принимает радианы.

5. Как безопасно выполнить действие через 2 секунды?

   (a) `sleep(2); action()`

   (b) `Timer.callLater(2, action)`

   (c) `Timer.new(2, action)`

   (d) `ap.delay(2, action)`

   Ответ: (b). Пояснение: отложенный неблокирующий вызов.

6. Как задать таблицу точек окружности?

   (a) `cos(i),sin(i)`

   (b) `r*math.cos(...), r*math.sin(...)`

   (c) `r*cos(...), r*sin(...)`

   (d) `x=i, y=i`

   Ответ: (b). Пояснение: вычисление по параметрическим формулам.

7. Как корректно выключить двигатели?

   (a) `ap.push(ENGINES_DISARM)`

   (b) `ap.push(Ev.ENGINES_DISARM)`

   (c) `ap.disarm()`

   (d) `ap.push(Ev.MCE_STOP)`

   Ответ: (b). Пояснение: требуется пространство имён `Ev`.

8. Как реагировать на удар/столкновение?

   (a) `event==Ev.SHOCK`

   (b) `event==Ev.COLLISION`

   (c) `event==Ev.CRASH`

   (d) `event==Ev.HIT`

   Ответ: (a). Пояснение: используйте `Ev.SHOCK`.

9. Как организовать непрерывное обновление точки окружности?

(a) `while ... do ap.goToLocalPoint(...) end`

(b) `Timer.new(period, function() ap.goToLocalPoint(...) end)`

(c) `repeat ap.goToLocalPoint(...) until false`

(d) `sleep(0.1); ap.goToLocalPoint(...)`

Ответ: (b). Пояснение: периодический таймер без блокировок.

10. Как прочитать RC-каналы?

    (a) `Sensors.rc()`

    (b) `ap.rc()`

    (c) `rc.read()`

    (d) `Sensors.readRC()`

    Ответ: (a). Пояснение: API сенсоров.

11. Как проверить положение тумблера SwA в `rc_chans`?

    (a) `rc_chans[1]`

    (b) `rc_chans[8]`

    (c) `rc_chans["SwA"]`

    (d) `rc_chans[0]`

    Ответ: (b). Пояснение: восьмой канал.

12. Как построить переходы состояний?

    (a) `if ... then ...`

    (b) таблица функций `action[state]()`

    (c) `switch(state)`

    (d) `goto state`

    Ответ: (b). Пояснение: идиоматичный подход через таблицу функций.

13. Как задать высоту полёта по окружности?

    (a) `ap.setHeight(z)`

(b) `ap.goToLocalPoint(x,y,z)`

(c) `ap.updateYaw(z)`

(d) `Sensors.altimeter(z)`

Ответ: (b). Пояснение: высота `z` задаётся в целевой точке.

14. Как обеспечить аварийную индикацию при низком напряжении?

    (a) в `callback` по `Ev.LOW_VOLTAGE2` остановить таймеры и включить красный

    (b) задержка через `sleep`

    (c) увеличить `height`

    (d) проигнорировать событие

    Ответ: (a). Пояснение: минимизация нагрузки и чёткая реакция.

3.12 Задания для лабораторной работы

3.12.1. Базовые миссии

1. **Полёт по окружности радиуса `r=0.5` на высоте `z=1.0`.** Требуется реализовать алгоритм, обеспечивающий устойчивый полёт по окружности: выполнить `ap.push(Ev.MCE_PREFLIGHT)` и `ap.push(Ev.MCE_TAKEOFF)`, запускать обновление точки через `Timer.new(0.1, fn)` после `Ev.TAKEOFF_COMPLETE`, вычислять `x=r*cos(theta)`, `y=r*sin(theta)` с переводом градусов в радианы; при необходимости сопровождать курс вызовом `ap.updateYaw(angle*math.pi/180)`; использовать индикацию `Ledbar.new(25)` для этапов; по `Ev.SHOCK` — останов таймеров и `ap.push(Ev.ENGINES_DISARM)`, по `Ev.LOW_VOLTAGE2` — посадку. *Ожидаемый результат:* квадрокоптер следует окружности радиуса `0.5` на высоте `1.0`, выполняет посадку, индикация отражает этапы.

2. **Полёт по многоугольнику из `N` точек окружности.** Требуется реализовать генерацию точек по равномерным углам `a=360*i/N`, переводить углы в радианы при вычислении координат, стартовать маршрут по `Ev.TAKEOFF_COMPLETE` и переходить к следующей точке по

`Ev.POINT_REACHED`; обеспечить безопасные реакции и индикацию как в окружности. *Ожидаемый результат:* последовательный обход всех `N` точек и посадка после завершения маршрута.

3. **Траектория «восьмёрка».** Требуется реализовать два состояния (`FIRST`, `SECOND`) для обхода двух окружностей с противоположным направлением, сместить центры по оси X, выбрать шаг угла разного знака для фаз, завершить миссию возвратом в `(0,0,z)` и посадкой. *Ожидаемый результат:* полёт по «восьмёрке» без рывков, завершение в центре и посадка.

4. **Спираль подъёма.** Требуется реализовать параметрическую спираль с приращениями `r=r+dr` и `z=z+dz` на каждом тике; по достижении `z_max` остановить таймер, перейти в безопасную точку базы и выполнить посадку. *Ожидаемый результат:* плавный подъём до `z_max`, возврат к базе и посадка.

5. **Сопровождение курса `yaw`.** Требуется синхронизировать `ap.updateYaw` с параметром окружности, использовать радианы; ограничить шаг изменения угла разумным значением, чтобы исключить резкие скачки. *Ожидаемый результат:* курс следует траектории без резких изменений, ориентация корректна.

6. **Запуск и останов миссии RC-тумблером.** Требуется считывать `Sensors.rc()`, анализировать `rc_chans[8]`; при включении тумблера выполнять предстарт и взлёт, при выключении — останавливаться, переходить в `(0,0,z)` и выполнять посадку; опрашивать RC через периодический `Timer.new(0.5, fn)`. *Ожидаемый результат:* надёжный старт/останов миссии по положению тумблера с безопасным завершением.

7. **Аварийная реакция на `Ev.SHOCK`.** Требуется немедленно остановить все таймеры миссии, включить красный цвет на всех `0..24` индексах линейки и выполнить `ap.push(Ev.ENGINES_DISARM)`. *Ожидаемый результат:* мгновенное выключение двигателей и красная индикация аварии.

8. **Посадка при низком напряжении `Ev.LOW_VOLTAGE2`.** Требует-

ся остановить миссию, перейти в безопасную точку и выполнить `ap.push(Ev.MCE_LANDING)`, включить красную индикацию низкого напряжения. *Ожидаемый результат:* безопасная посадка при низком напряжении, отчёт индикацией.

9. **Обход квадрата по точкам, два цикла.** Требуется задать вершины `(-a/2,-a/2,z)`, `(a/2,-a/2,z)`, `(a/2,a/2,z)`, `(-a/2,a/2,z)`, выполнять переходы по `Ev.POINT_REACHED` и совершить два полных обхода перед посадкой. *Ожидаемый результат:* два полных обхода квадрата и посадка в конце маршрута.

10. **«Лоитер» по малой окружности `r=0.2`.** Требуется периодически обновлять точку окружности таймером, обеспечить безопасную остановку по событию или RC, сопровождать миссию световой индикацией этапов. *Ожидаемый результат:* стабильное удержание по малой окружности и корректная остановка по команде.

3.12.2. Продвинутые задания

1. **Плавная смена цвета этапов миссии.** Требуется сопоставить цвета состояниям миссии (подготовка/полёт/посадка/ожидание), обновлять все индексы `0..24` без блокировок `sleep()`, реализовать смену через функции стадий и отложенные вызовы. *Ожидаемый результат:* плавная индикация переходов между этапами без пропусков и блокировок.

2. **Динамический радиус окружности по RC7.** Требуется считывать `Sensors.rc()`, нормировать значение канала к `[0,1]`, вычислять радиус `r=0.2+0.8*norm` и периодически обновлять точку окружности. *Ожидаемый результат:* радиус плавно меняется от `0.2` до `1.0` по RC7, траектория стабильна.

3. **Ограничение скорости изменения угла `angle`.** Требуется реализовать `advanceAngle(step)` с ограничением `step` диапазоном `[-maxStep,maxStep]` и применять его при каждом обновлении параметра окружности. *Ожидаемый результат:* угол изменяется с ограниченной скоростью, резкие скачки отсутствуют.

4. **Перезапуск миссии после `Ev.COPTER_LANDED`.** Требуется по событию

выполнить `Timer.callLater(3, fn)` и инициировать новый взлёт, исключая блокирующие задержки. *Ожидаемый результат:* автоматический повтор миссии после посадки с паузой, без блокировок.

5. **«Морзянка» этапов на линейке.** Требуется реализовать мигание заданное число раз для каждого этапа, используя короткие функции и отложенные вызовы. *Ожидаемый результат:* корректный световой отчёт по морзянке для каждого этапа.

6. **Траектория «лепесток».** Требуется реализовать формулу `r=r0*(1+0.2*sin(k*theta))` при `theta=angle*pi/180`, выбрать `k>=3` и обновлять точку параметрически. *Ожидаемый результат:* визуально различимый «лепесток» по параметрической траектории.

7. **Проверка точности навигации.** Требуется обход 8 точек окружности, измерение интервалов `Δt` между `Ev.POINT_REACHED` с использованием `os.clock()`. *Ожидаемый результат:* получены интервалы времени, оценено постоянство периода между достижениями точек.

8. **Миссия «периметр» вокруг базы.** Требуется определить набор точек по квадрату вокруг `(0,0)`, включить возврат в исходную точку и выполнить посадку. *Ожидаемый результат:* полный обход периметра и посадка в исходной точке.

9. **«Окно безопасности» для `yaw`.** Требуется ограничить изменение `yaw` на тик не более `maxYawRate*dt` и обновлять точку только при соблюдении ограничения. *Ожидаемый результат:* курс изменяется не быстрее установленной скорости, миссия сохраняет устойчивость.

10. **Демонстрационный сценарий (RC старт, ≈ 2 минуты, посадка).** Требуется запуск по RC, полёт по окружности около двух минут, посадка, световой отчёт этапов, отсутствие блокировок и проверка безопасного завершения. *Ожидаемый результат:* демонстрационный полёт по окружности, корректная посадка и полная индикация этапов.